

bauds

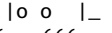
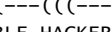
ISSUE No 1 – Oct 2025

9600@ghspace.fr
#9600 sur irc.libera.chat

			
			
: 1	: 1	: 1	: 1
:... .	:... .	:... .	:... .

bauds

ISSUE No 1 - Oct 2025




GRENOBLE HACKERSPACE

9600@ghspace.fr
#9600 sur irc.libera.chat

-- ABOUT --

GRENOBLE HACKERSPACE est une association loi 1901, un tier-lieu et une communauté dédiée à la promotion des arts, de l'expérimentation, l'informatique et la cybersécurité. L'association offre un espace de rencontre et de travail pour apprendre, bricoler, collaborer, détourner ensemble.

-- FAQ --

Q : Faut-il être expert.e pour venir ?

A : Non ! L'idée est justement d'apprendre ensemble. Seuls prérequis : curiosité et respect du lieu.

Q : Puis-je venir de manière ponctuelle ?

A : Bien sûr ! Les Apéroots et certains ateliers sont ouverts à tou·tes.

Q : Combien ça coûte ?

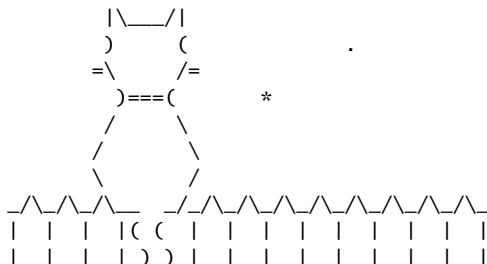
A : Le prix est libre mais un abonnement de 16€/mois est recommandé.

Q : Qui prend les décisions ?

A : Chaque décision est validée collectivement par ces membres actifs, directement ou pendant les assemblées.

Q : C'est légal ?

A : Tout ce qui est fait dans le cadre du lieu respecte la loi. Le reste, cela ne nous regarde pas !



-- EOF --

SUMMARIZE

-- INFO --

9600 bauds est un ezine inter-hackerspaces francophones à l'initiative de GRENOBLE HACKERSPACE. Vous pouvez partager vos projets, vos actualités, vos réflexions, vos tutoriels, vos retours d'expérience, ou tout autre contenu de votre communauté dans un format volontairement minimaliste et nostalgique

SOMMAIRE	
00.TXT : Sommaire et Présentations	Staff 9600
01.TXT : Radio Libre, radio pirate!	Beemo
02.TXT : RADIO TOC TOC : une sonnette pour ton bat'	adada
03.TXT : Hpphckngref: C&Embedded quick refs	tixlegeek
04.TXT : wall-E: sueur, fils et plastique fondu	cryptax
05.TXT : Fuck around, Mess Around, 10 ans de *GBO*	Cha-0s & iooner
06.TXT : Surveillance et narcotraficotage	Beemo

_____ +
 / \
 _____ / o \ \ _____

| o o o | _____ | _____ | | # # # | _____ | o o o o | / \
 | o o o | * * * | : : | . . | | o | # # # | . . | o o o o | / \ \ \
 | o o o | * * * | : : | . . | [] [] [] [] | o | # # # | . . | o o o o | ((())
 | o o o | ** ** | : : | . . | [] [] [] [] | o | # # # | . . | o o o o | ((())
 | [] | _____ | [] | | | | < | _____ | : : | _____ | ^ | _____ | [] | _____

```
-- EOF --
```



-- EDITO --

Bonsoir !

C'est avec joie et nostalgie que je vous présente enfin le tout premier numéro de "9600 bauds". Pour tout vous dire, ce projet n'était pas censé sortir de ma tête, une idée de soirée, effectivement un peu trop arrosée. Mais nous y voilà ! Et franchement, ça fait du bien. Les petits (et gros) bouts d'articles que vous allez découvrir viennent de Grenoble, Sophia, Liège, du Marais Poitevins... Bref, du contenu par et pour nous, sans filtre, sans pub, en mode TXT.

Un immense merci à ceux qui ont contribué dès le début, ou à la dernière minute. On espère que ce premier numéro vous plaira et vous motivera pour les prochains ;)

Biz.

Beemo

-- EOF --

Radio libre, radio pirate!

Noë Flatreaud <nflatrea@mailo.com> (Beemo)
GRENOBLE HACKERSPACE (Grenoble)

-- CONTENT --

C'est vrai ça, pourquoi qu'on c'est mis à détourner des fréquences ?

Entre 1960 et 1970, les medias traditionnels étaient (et sont encore) fermement contrôlés par l'État et certaines entreprises. Les fréquences distribuées et définies par l'État étaient fortement surveillées et tout mésusage banni.

La radio pirate est donc née d'un besoin de contre-pouvoir et de dissidence. En Italie, Radio Alice [^1] balançait des débats anarchistes et des concerts punk. En France, les radios syndicales comme Radio Riposte [^2] donnaient la parole aux grévistes pendant que les flics tentaient de les localiser. Pas de pubs, pas de patrons, pas de playlist imposée. Juste des vrai gens qui parlaient à d'autres vrai gens. Les studios étaient souvent improvisés : Des caves, des camions, voir même des bateaux (Radio Caroline [^3]).

Goooooooood morning england!

```
      |   |   |   /
      )_) )_) )_)
      )_) )_) )_) \
      )_) )_) )_) \
      _____ \
-----\           /-----
  ^^^^^ ^^^^^^^^^^^^^^^^^^^^^^^^^
    ^^^  ^^^^      ^^^  ^^
      ^^^  ^^^
```

-- Déclin

Mais bon, l'âge d'or n'as pas duré,

En 67, au Royaume-Uni, le Marine Broadcasting Offences Act envoie les pirates en taule. En France, les flics confisquent les émetteurs, les studios sont

perquisitionnés, les animateurs traînés. Radio Riposte ? Fermée. Radio Tomato [Λ4] ? Saisie. À partir des années 80, les radios pirates ont été traquées, étouffées, remplacées. D'abord par les radios libres (autorisées en 1982), puis par les radios privées (la thune!).

En parallèle, l'arrivée d'Internet marque un tournant, plus simple et moins risqués, les gens ont commencé à se tourner vers des proto-plateformes pour diffuser et écouter de la musique. Les webradios offraient une alternative presque légale et plus accessible, permettant à quiconque de créer sa propre station.

La manière dont on consomme a aussi beaucoup changé, avec des plateformes comme Napster [A5], il était largement plus simple de télécharger puis écouter la musique hors-ligne plutôt que de l'entendre boquer pendant 45 min.

Après, bonne nouvelle : les radios pirates n'ont pas totalement disparu. Elles ont mutés. Certaines utilisent les ondes courtes (60, 49, 41 ou 31 mètres) pour toucher des régions entières sans se faire choper.

D'autres émettent en portable, une heure par mois, avant de disparaître. En Europe, certains pays ferment les yeux : Pays-Bas, Belgique, Danemark. Tant que les pirates ne brouillent pas les fréquences officielles, on les laisse faire leur tambouille. Aussi, les mouvements libertaires, écologistes et LGBTQI+ se sont appropriés cette méthode, avec un retour rapide pour détourner certaines émissions de fachos ou informer la populace (Radio Klaxon sur la 107.7 de Notre-Dames-Des-Landes ou Radio Debout Place de la République)

Bref, la radio c'est cool, qu'elle soit légale ou pas, elle reste un bon moyen de communication et de diffusion. Des initiatives locales et communautaires continuent de fleurir, au plus grand bonheur des petites gens.

Biz.

-- Références

[^1] https://en.wikipedia.org/wiki/Radio_Alice

[^2] https://fr.wikipedia.org/wiki/Radio_Riposte

[^3] https://en.wikipedia.org/wiki/Radio_Caroline

[^4] https://fr.wikipedia.org/wiki/Radio_Tomate

[^5] <https://en.wikipedia.org/wiki/Napster>

[A6] <https://www.socialter.fr/article/le-retour-des-radios-pirates-podcasts-resistance-liberte-expression>

```
-- EOF --
```

RADIO TOC TOC

une sonnette pour ton bat'

Ada LaDigue (adada)
L'Inondation (marais poitevin)

-- CONTENT --

```

19:47 <hexa> euh c ki ki a mis Philippe Lavigne en boucle sur radio toc toc ??  
19:49 <heidi> /o\ tu peux changer hein ! J'ai mettre radio vnr allez  
19:50 <heidi> <https://jukebox.local> (si t'es co au wifi ou au VPN)  
19:52 <hexa> MERCI  
\* TULUTUTULUTU TU \*  
19:58 <heidi> ah jv ouvrir  
```

Besoin d'une sonnette pour votre bat'/hangar ?

Installez RADIO TOC TOC !

- 1 émetteur FM bien placé dans le bâtiment
- 1 routeur wifi (VPN facultatif)
- 1 bouton poussoir sur la porte du bat' (clic)
- 1 ordi avec Moode ou un autre audio avec interface web pour passer du son
- 1 arduino ou ESP32-like qui joue une jolie mélodie quand le bouton est appuyé
- 1 mixeur de son passif (quelques résistances et condo)
- du fil électrique

-- EOF --

Hpphckngref

C & Embedded quick refs

tixlegeek
HZV (Hackerzvoice)

-- CONTENT --

-- 2.1 C

tixlegeek's  1.1
C QUICK REFERENCE (GCC, C89/C99, LP64)

sizeof(void*)=8, sizeof(long)=8, sizeof(int)=4.
Impl-def:implementation-defined.
UB = undefined behavior.

TYPES & SIZES		STORAGE / QUALIFIERS	
char	1 (signed?/impl-def)	const	> read-only view
signed char ...	1 (-128..127)	volatile	> value may change externally
unsigned char .	1 (0..255)	restrict	> sole pointer (C99, opt hint)
short	2 (≥16 bits)	static	> internal linkage/static dur.
int	4 (≥16 bits)	extern	> external linkage
long	8 (≥32 bits)	auto	> default local storage
long long	8 (≥64 bits, C99)		
float	4 IEEE754 single	ALIGNMENT	
double	8 IEEE754 double		
long double ...	16* x87 ext (impl)	alignof(T)	= required alignment
_Bool / bool ..	1 (C99,)	alignas(n)	= force alignment
void*	8 (generic pointer)	GCC: __attribute__((aligned(n)))	
size_t	8 (unsigned index)	Packing (non-portable):	
ptrdiff_t	8 (signed ptr diff)	#pragma pack(push,1) ... #pragma pop	
int8_t,int16_t..	intptr_t	GCC: __attribute__((packed))	

STRUCTS & BITFIELDS	ENUMS & TYPEDEF
<pre> struct P{int x,y}; sizeof(struct) ≥ sum, includes pad. Reorder fields to reduce padding. Flexible array (C99): struct Buf{size_t n; char d[]}; p=malloc(sizeof*p+n); Bitfields (impl-def layout!): struct F{ unsigned rdy:1; unsigned mode:3; signed off:12; unsigned :0;}; </pre>	<pre> enum Color{RED, GREEN, BLUE}; enum Err{OK=0, IO=5, RANGE=100}; Underlying type=impl-def (usually int). Tip: add *_COUNT for bounds. typedef unsigned long ulong; typedef int(*cmp)(const void*, const void*); typedef struct Node Node; // opaque handle </pre>
BITWISE OPS	MEMORY MGMT ()
<pre> & AND OR ^ XOR ~ NOT << shL >> shr Prec: ~ > << >> > & > ^ > #define BIT(n) (1u<<(n)) flags = BIT(3); // set flags &= ~BIT(3); // clear if(flags & BIT(3)) ... Shift ≥width or <0 => UB Right shift signed neg=impl-def </pre>	<pre> malloc(n) uninitialized calloc(c,s) zeroed realloc(p,n) resize, may move free(p) release Pattern: int *a=malloc(n*sizeof*a); int *t=realloc(a,n2*sizeof*a); if(t) a=t; </pre>
ARRAYS & POINTERS	FUNCTIONS & POINTERS
<pre> int a[3]={1,2,3}; int *p=a; // decay to pointer sizeof a=3*4=12, sizeof p=8 VLA (C99): void f(n){int vla[n];} </pre>	<pre> typedef int(*pred)(int); int even(int x){return !(x&1);} int apply(pred f, int *arr, size_t n){...} CONST POINTERS const int *p; > ptr to const int *const q; > const ptr const int *const r; > const ptr to const </pre>
UNIONS	GCC EXTENSIONS
<pre> union U{uint32_t u; unsigned char b[4];}; u.u=0xAABBCCDD; Note: reading inactive member is impl-def. Use memcpy for type pun. </pre>	<pre> __attribute__((aligned(n))) force align __attribute__((packed)) no padding __attribute__((unused)) suppress warn __attribute__((noreturn)) func never ret __attribute__((format(printf,...))) __builtin_expect(x,1) > branch prediction </pre>

MACROS / DEFINES	COMMON UB (Undefined Behavior)
#define MAX(a,b) ((a)>(b)?(a):(b))	- signed int overflow
#define MIN(a,b) ((a)<(b)?(a):(b))	- shift by <0 or ≥width
#define LEN(a)(sizeof(a)/sizeof*(a))	- out-of-bounds pointer arithmetic
	- use-after-free / double free
	- modify string literal
	- strict aliasing violations(expt char*)
	- read inactive union member

-- 2.2 COMPILATION

tixlegeek's



1.2

COMPILATION & LINKING QUICK REFERENCE (C / GCC)

COMPILATION PHASES	COMMANDS / FLAGS
1. Preprocess: expand #include, macros (#define, #ifdef)	gcc -E file.c -o file.i (stop at cpp) gcc -S file.i -o file.s (stop at asm)
2. Compile: C > assembly	gcc -c file.s -o file.o (stop at obj)
3. Assemble: asm > object (.o)	gcc file.o -o prog (link to exe)
4. Link: resolve symbols, libs, produce executable / lib	gcc *.c -o prog (all steps) gcc main.c -L. -lmylib -lm (link libs)

STARTUP / RUNTIME	LIBRARIES
crt0 / crt1.o: runtime startup	Static lib (.a):
- sets up stack, TLS	ar rcs libfoo.a foo.o bar.o
- calls __libc_start_main()	gcc main.o -L. -lfoo -o app
- runs global constructors	
- calls main(), then exit()	Shared lib (.so):
	gcc -fPIC -c foo.c
	gcc -shared -o libfoo.so foo.o
	gcc main.o -L. -lfoo -Wl,-rpath=. -o app

SYMBOL RESOLUTION	SYMBOL TOOLS
Undefined > must be resolved at link	nm libfoo.a list symbols
Defined in .o or library?	nm -D libfoo.so dynamic symbols
First found wins (link order!)	objdump -d prog disassembly
Duplicate definitions > error	objdump -t prog symbol table
	readelf -h prog ELF header
	readelf -s prog symbols
	ldd prog show shared libs used

LINKER FLAGS (with -Wl,)	GCC-LD COMMON OPTIONS
-Wl,-Map,link.map generate map	-nostdlib skip crt, libc, stdlib
-Wl,--gc-sections drop unused	-nodefaultlibs skip stdlibs only
-Wl,-rpath,path runtime search	-nostartfiles skip crt0

-Wl,-rpath-link,p link-time search	-Wl,--as-needed only pull needed libs
-Wl,-z,defs no unresolved	-Wl,-static fully static binary
-Wl,-z,noexecstack secure stack	-Wl,-dynamic-list=listfile
-Wl,--whole-archive link full .a	-Wl,-O1 optimize linker
-Wl,--start-group ... --end-group	-Wl,-soname,libfoo.so.1 (for .so)

ORDER MATTERS (STATIC LINKING)

```
gcc main.o lib1.a lib2.a -o app (OK)
gcc lib1.a lib2.a main.o -o app (MAY FAIL, symbols unresolved)
```

Tip: libs should follow objects that need them.

LINKER SCRIPTS (.ld) | MEMORY LAYOUT (ELF)

ENTRY(_start);	Sections:
SECTIONS {	.text code
.text : { *(.text) }	.rodata const data
.rodata : { *(.rodata*) }	.data init RW data
.data : { *(.data*) }	.bss zero-init data
.bss : { *(.bss*) *(COMMON) }	.init, .fini startup/teardown hooks
}	.plt/.got dynamic linking tables
	.stack, .heap (runtime allocated)

STATIC VS DYNAMIC | POSITION INDEPENDENT

Static:	-fPIC for shared libs (position-indep)
+ self-contained binary	-fPIE for executables (ASLR)
+ no external deps	-pie link as position-indep exe
- larger, no sec updates to libs	
Dynamic:	Check:
+ smaller, updates auto (libc.so)	readelf -h prog grep Type
+ share memory pages	DYN > PIE
- requires runtime libs installed	EXEC > fixed exe

DEBUG INFO & STRIPPING | MULTI-FILE PROJECTS

-g generate DWARF	gcc -c file1.c file2.c
-ggdb gdb friendly	ar rcs libfoo.a file1.o file2.o
-strip remove debug info	gcc main.c -L. -lfoo -o app
-Os -s (optimize & strip)	
objcopy --only-keep-debug app dbg	Makefiles / CMake handle dep & linking.
objcopy --strip-debug app	
gdb ./app dbg	

EXAMPLES

```
# Build > obj > link
$ gcc -c main.c util.c
$ ar rcs libutil.a util.o
$ gcc main.o -L. -lutil -o app
```

```
# Shared lib
$ gcc -fPIC -c foo.c
$ gcc -shared -o libfoo.so foo.o
$ gcc main.c -L. -lfoo -Wl,-rpath=. -o app

# Inspect symbols
$ nm libfoo.a
$ nm -D libfoo.so
$ readelf -s app | grep foo

# Custom linker script
$ arm-none-eabi-ld -T stm32.ld -o fw.elf crt0.o main.o

# Position-independent exe
$ gcc -fPIE -pie main.c -o app
```

-- 2.3 EMBEDDED

tixlegeek's



EMBEDDED C QUICK REFERENCE (GCC / BARE-METAL / RTOS)

TOOLCHAIN / CROSS COMPILE	MEMORY MODEL
arm-none-eabi-gcc ...	.text code (flash/ROM)
avr-gcc ...	.rodata const data (flash/ROM)
mipsel-elf-gcc ...	.data init RW data (RAM)
riscv64-unknown-elf-gcc ...	.bss zero-init vars (RAM)
Flags:	stack local vars, calls
-mcpu=cortex-m3 -mthumb	heap malloc/free (avoid on small MCU)
-mmcu=atmega328p	Linker script (.ld) maps regions
-nostdlib -nostartfiles	MEMORY MAP = critical (flash size, RAM)
-T stm32.ld	
STARTUP / INIT	ISR (INTERRUPT SERVICE ROUTINE)
Reset handler runs before main():	void __attribute__((interrupt))
- Set stack pointer	isr(void) { ... }
- Copy .data from flash>RAM	
- Zero .bss	Cortex-M (CMSIS):
- Call constructors (C++)	void SysTick_Handler(void){ ... }
- Jump to main()	
	AVR (avr-gcc):
Provided by: crt0, startup.s	ISR(TIMER1_COMPA_vect){ ... }
BITWISE MACROS (PERIPHERALS)	INLINE & OPTIMIZATION
#define BV(n) (1u<<(n))	-O0 debug, -Og dev debug
#define SET(reg,n) ((reg) =BV(n))	-O2 common release
#define CLR(reg,n) ((reg)&=~BV(n))	-Os optimize for size (MCU)

#define TOG(reg,n) ((reg)^=BV(n))	-flto link-time optimization
#define TST(reg,n) ((reg)&BV(n))	-fdata-sections -ffunction-sections
	+ -Wl,--gc-sections > shrink binary
DEBUGGING & TESTING	HARDWARE DEBUG INTERFACES
printf > large; use UART minimal Semihosting (debug print via gdb)	SWD (ARM), JTAG (common), ISP (AVR)
LED blink > "hello world"	OpenOCD + GDB:
UART logs: ring buffer + ISR	arm-none-eabi-gdb prog.elf
	target remote :3333
	load / monitor reset / continue
Unit tests: simulate on host (QEMU)	
GCC EMBEDDED FLAGS	HARDENING & SAFETY
-fno-builtin -ffreestanding	-fstack-protector-strong (if RTOS)
-fshort-enums (size saving)	-D_FORTIFY_SOURCE=2 (needs libc)
-mcpu=... -mthumb (ARM Thumb mode)	-Wcast-align -Wvolatile-register-var
-fomit-frame-pointer (size)	MISRA-C rules > safety coding std
	Avoid dynamic mem in safety-critical

-- 2.4 GDB

tixlegeek's



GDB QUICK REFERENCE (Linux)

Build for debug (recommended):
gcc -g -Og -fno-omit-frame-pointer -Wall -Wextra -o app app.c

STARTING / ATTACHING	BASIC RUN/STEP	
gdb ./app	run [args]	start program
gdb ./app core	start	run to main
gdb -q --args ./app a b c	continue (c)	resume
(gdb) set args a b c	next (n)	step over
(gdb) run < in.txt > out.txt	step (s)	step into
attach PID	finish	run until ret
detach	until loc/line	run until location
kill	advance func	run to func entry
quit	stepi/nexti	single-insn
BREAKPOINTS	WATCHPOINTS	
break main	watch expr	stop on write
break file.c:123	rwatch expr	stop on read
break func if x>3	awatch expr	stop on access
tbreak func (temp)	info watchpoints	
hbreak func (hw bp)	delete N / disable N / enable N	
rbreak regex (many)	cond N expr	set condition
info breakpoints / delete N	(hw wps are limited; prefer scalars)	

PRINTING & TYPES		MEMORY EXAMINE (x/)
print x	(p x)	x/
p/x x	hex	fmt: x hex d dec u uns o oct t bin
p/d x	signed decimal	a addr c char s string i insn f
float		
p/t x	binary	size: b byte h halfword w word g giant
p/a ptr	address	ex: x/16xb buf 16 bytes in hex
ptype var	C type	x/8i \$pc 8 instruct at \$pc
whatis var	brief type	x/3a &arr 3 addresses
set print pretty on (structs)		display/i \$pc auto-show each step
set print elements 0 (no limit)		set disassemble-next-line on
set print array on		
set print address on/off		
STACK / FRAMES		SOURCE / SYMBOLS
backtrace (bt)		list show source
bt full (locals too)		list file.c:120 around line
frame N / up / down		info source / info sources
info args / info locals		info functions regex
info registers / p \$rax		directory src/ add src paths
set \$pc=expr (jump)		file ./app set exec+symbols
return (force return)		symbol-file symfile sym only
THREADS / PROCESSES		SIGNALS & EXCEPTIONS
info threads		info signals
thread N		handle SIGPIPE nostop noprint pass
thread apply all bt		handle SIGSEGV stop print
set scheduler-locking step/on/off		catch throw/catch (C++)
set follow-fork-mode child parent		catch syscall open,write,...
set detach-on-fork off		catch fork vfork exec load unload
info inferiors / inferior N		signal SIGUSR1 deliver to program
INPUT/ENV/WD		LOGGING & SCRIPTS
set args ...		set pagination off
show args		set logging on/off file gdb.log
run < in > out 2>err		source script.gdb
set environment K=v		define cmd ... end user command
unset environment K		document cmd ... end help text
cd path / pwd		define hook-stop ... auto after stop
tty /dev/pts/3 (program's TTY)		alias nni = nexti; etc.
SHARED LIBS		CORE DUMPS
info sharedlibrary		ulimit -c unlimited enable cores
set breakpoint pending on		gdb ./app core
break libfn (before lib loads)		bt / bt full
set solib-search-path ./lib:...		thread apply all bt full
set sysroot /path/to/sysroot		info threads / info registers
set debuginfod enabled on (if avail)		frame N; info locals; x/... mem

DISASSEMBLY / ASM		TUI (text UI)	
disassemble /m func (with source)		tui enable	
disassemble 0xADDR,+LEN		layout src layout asm layout split	
x/i \$pc (one instruction)		Ctrl-x a toggle TUI	
display/i \$pc (auto each step)		refresh redraw	
set disassemble-next-line on			
ADVANCED BREAKPOINTS		BREAKPOINT COMMAND LISTS	
break file.c:123 thread 5		break foo	
break file.c:123 if i==42		commands	
ignore N COUNT (skip N, COUNT times)		silent	
condition N expr (change later)		printf "i=%d\n\n", i	
commands N ... end (scripted)		bt 2	
bp enable/disable temporary		continue	
save breakpoints bps.txt		end	
source bps.txt (reload)			
REMOTE DEBUGGING		REVERSE / CHECKPOINTS	
On target: gdbserver :1234 ./app sup)		record enable recording (if	
On host:		reverse-continue / reverse-(step stepi)	
gdb ./app		checkpoint snapshot process	
(gdb) target remote :1234		info checkpoints / restart N	
(gdb) set sysroot /sysroot			
(gdb) set solib-search-path ...			
(gdb) continue			
CALLING & MODIFYING STATE		PRACTICAL PRINT TRICKS	
call fn(42)		p/x *(uint32_t*)buf	32-b value at buf
set var x=5 (or just: set x=5)		p (int)strchr(s,'x')-s	index of 'x'
set {int}0xADDR = 0		p *(struct S*)ptr	dump struct
set \$sp = \$sp-16		set print pretty on	nicer structs
set \$pc = 0xADDR (jump)		set print elements 0	no trunc arrays
FORMAT & EXAMINE CHEATSHEET			
x/16xb p	16 bytes hex from p	x/20cb p	20 chars
x/8xh p	8 halfwords (16-bit) hex	x/4xw p	4 words hex
x/6i \$pc	6 instructions at PC	p/x arr[i]	hex print element
x/s p	C string at p	p/t v	binary
x/a p	address format	p/f d	print float/double
help x	(for all modifiers)	help p	(formats for print)
OPTIMIZATION & REALITY			
- -Og keeps debug friendly; -O2 may inline/remove vars > use `info scope`. - Use `disassemble /m`, `info registers`, and watchpoints to reason at -O2. - `set disassemble-next-line on` helps step source+asm coherently.			

GOOD HABITS

- Start with: set pagination off; set confirm off; set print pretty on
 - Use conditional breakpoints instead of manual stepping.
 - Use `thread apply all bt` early when analyzing crashes.
 - Prefer `sigaction` & handle SIGPIPE/SIGCHLD in the program for sanity.
 - Log gdb session: set logging on; set logging overwrite on
 - Keep a project .gdbinit with your defaults & helper commands.
-

MINI .gdbinit (drop in project root)

```
set pagination off
set confirm off
set print pretty on
set print elements 0
set disassemble-next-line on
define bpc          # break on call to function by name
    rbreak ^$arg0$
end
define btall
    thread apply all bt full
end
define nl           # next N lines
    list
    list
end
```

REAL-WORLD WORKFLOWS

```
# Debug a crash via core:
$ ulimit -c unlimited && ./app
$ gdb ./app core
(gdb) bt full
(gdb) thread apply all bt full
(gdb) frame 3; info locals; x/32xb buf

# Attach production process, snapshot core without stopping long:
$ gdb -q -p PID
(gdb) gcore dump.core
(gdb) detach; quit
$ gdb ./app dump.core

# Remote (ARM target):
target$ gdbserver :2345 ./app
host$  gdb ./app
(gdb)  set sysroot /opt/arm-sysroot
(gdb)  target remote :2345
(gdb)  b main; c

# Break when a file is opened (Linux):
(gdb) catch syscall openat
(gdb) r
(gdb) bt
```

```
# Data race/odd segfault at -O2 > use watchpoint:
(gdb) watch *(int*)ptr
(gdb) c
```

-- 2.5 Appendix

tixlegeek's



VARIOUS QUICK REFERENCE (Linux / Embedded)

COMMON BAUD RATES

Standard (HW-supported)			High / Non-standard (USB/FTDI etc.)		
300	600	1200	230400	250000	256000
2400	4800	9600	460800	500000	576000
14400	19200	28800	921600	1000000	1152000
38400	57600	115200	1500000	2000000	3000000

TYPICAL FRAME FORMAT	SIGNALS / LINES	
1 start bit	TXD	transmit data
5-9 data bits	RXD	receive data
parity: none / even / odd	RTS	request to send
1-2 stop bits	CTS	clear to send
async, no clock	DTR	data terminal ready
	DSR	data set ready
	DCD	carrier detect
	RI	ring indicator

C EXAMPLE (TERMIO)

```
#include <termios.h>
#include <unistd.h>
#include <fcntl.h>
#include <stdio.h>

int main(void) {
    int fd = open("/dev/ttyUSB0", O_RDWR | O_NOCTTY);
    struct termios tty;
    tcgetattr(fd, &tty);

    cfsetospeed(&tty, B115200);
    cfsetispeed(&tty, B115200);
    tty.c_cflag = (tty.c_cflag & ~CSIZE) | CS8; // 8 data bits
    tty.c_cflag |= CREAD | CLOCAL; // enable RX, ignore modem
    tty.c_lflag = 0; // no canonical mode
    tty.c_iflag = 0; // raw input
    tty.c_oflag = 0; // raw output
    tty.c_cc[VMIN] = 1; // block for 1 char
    tty.c_cc[VTIME] = 0;
    tcsetattr(fd, TCSANOW, &tty);
```

```

write(fd, "AT\r\n", 4);
char buf[64];
int n = read(fd, buf, sizeof(buf));
write(STDOUT_FILENO, buf, n);
close(fd);
}

```

BASIC TERMINAL COMMANDS		DEBUG / LOGGING
# List devices		# Watch raw bytes
\$ ls /dev/ttyS* /dev/ttyUSB*		\$ hexdump -C /dev/ttyUSB0
# Show settings		# Log session
\$ stty -F /dev/ttyUSB0 -a		\$ script uart.log
# Configure		# Minimal raw mode
\$ stty -F /dev/ttyUSB0 115200 cs8 \		\$ stty raw -echo
-cstopb -parenb -echo		
MODEM / AT COMMANDS		RS-232 / TTL LEVELS
AT	check / OK	RS-232: ±12 V (inverted)
ATI	info	TTL-UART: 3.3 V or 5 V (non-inverted)
ATZ	reset	
AT+GMR	firmware version	
ATE0/1	echo off/on	
AT+RST	reset (ESP-type)	

ASCII TABLE (7-bit standard)

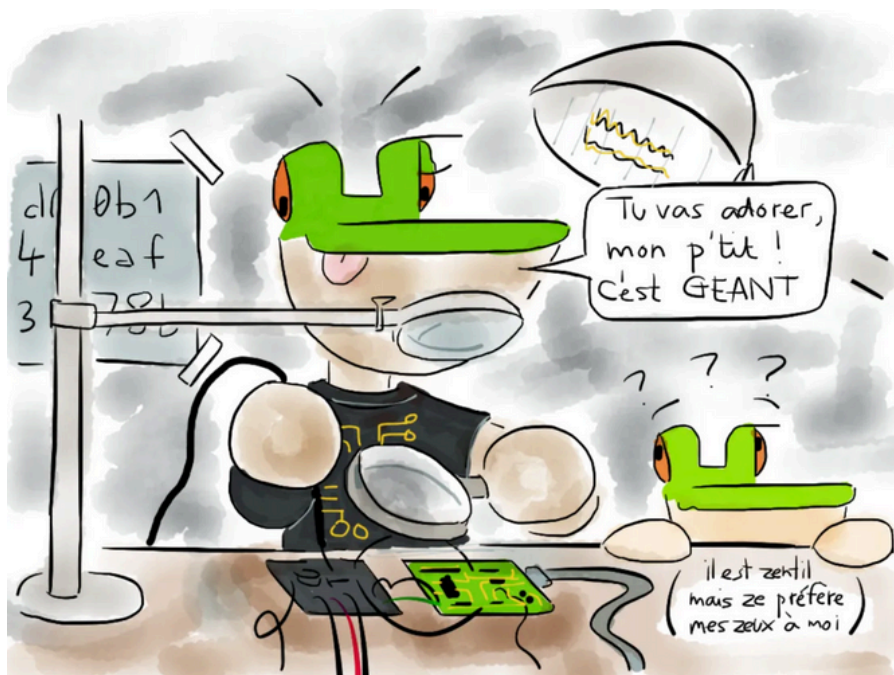
Dec	Hex	Chr	Dec	Hex	Chr	Dec	Hex	Chr	Dec	Hex	Chr	Dec	Hex	Chr	Dec	Hex	Chr
0	00	NUL	16	10	DLE	32	20	SPC	48	30	0	64	40	@	80	50	P
1	01	SOH	17	11	DC1	33	21	!	49	31	1	65	41	A	81	51	Q
2	02	STX	18	12	DC2	34	22	"	50	32	2	66	42	B	82	52	R
3	03	ETX	19	13	DC3	35	23	#	51	33	3	67	43	C	83	53	S
4	04	EOT	20	14	DC4	36	24	\$	52	34	4	68	44	D	84	54	T
5	05	ENQ	21	15	NAK	37	25	%	53	35	5	69	45	E	85	55	U
6	06	ACK	22	16	SYN	38	26	&	54	36	6	70	46	F	86	56	V
7	07	BEL	23	17	ETB	39	27	'	55	37	7	71	47	G	87	57	W
8	08	BS	24	18	CAN	40	28	(	56	38	8	72	48	H	88	58	X
9	09	HT	25	19	EM	41	29	)	57	39	9	73	49	I	89	59	Y
10	0A	LF	26	1A	SUB	42	2A	*	58	3A	:	74	4A	J	90	5A	Z
11	0B	VT	27	1B	ESC	43	2B	+	59	3B	;	75	4B	K	91	5B	[
12	0C	FF	28	1C	FS	44	2C	,	60	3C	<	76	4C	L	92	5C	\
13	0D	CR	29	1D	GS	45	2D	-	61	3D	=	77	4D	M	93	5D	]
14	0E	SO	30	1E	RS	46	2E	.	62	3E	>	78	4E	N	94	5E	^
15	0F	SI	31	1F	US	47	2F	/	63	3F	?	79	4F	O	95	5F	_

96 60 `	97 61 a	98 62 b	99 63 c	100 64 d	101 65 e
102 66 f	103 67 g	104 68 h	105 69 i	106 6A j	107 6B k
108 6C l	109 6D m	110 6E n	111 6F o	112 70 p	113 71 q
114 72 r	115 73 s	116 74 t	117 75 u	118 76 v	119 77 w
120 78 x	121 79 y	122 7A z	123 7B {	124 7C	125 7D }
126 7E ~	127 7F DEL				

PRACTICAL NOTES

- Default serial: 115200 8N1 (8 data, No parity, 1 stop).
- For binary data, use raw mode: ``stty raw -echo``.
- Log sessions: ``script serial.log``.
- Loopback test: short TX+RX, verify echo.
- Check user perms on ``/dev/ttyUSB*`` (group ``dialout``).

-- EOF --



@cryptax

wall-E : Sueur, fils et plastique fondu

Axelle Apvrille (cryptax)
Sophia Hack Lab (Sophia Antipolis)

-- CONTENT --

Il y a quelques mois, je me suis embarquée dans la construction d'un robot wall-E. Dans le film d'animation (2008), c'est un robot cubique qui est chargé de nettoyer la Terre de ses déchets, vous vous rappelez ?

J'ai choisi un projet conçu par "jeje95". Le robot est ressemblant, il parle, allume une ampoule et sert de réveil sur une table de chevet. C'est surtout un objet de décoration geek, pour le plaisir. J'ai pour objectif d'y mettre ma petite touche perso, avec quelques fonctionnalités de mon cru, comme l'affichage de la météo de notre station météo locale ou l'affichage de quelques photos du célèbre Pico le Croco (<https://pico.masdescrocodiles.fr>).

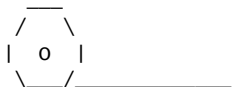
Un projet simple, documenté et réfléchi puisque Jeje95 fournit une vidéo de montage sur YouTube [VIDEO], des plans d'impression 3D sur Thingiverse [WALL1], une liste de composants [JEJE] et du code open source sur GitHub [CODE]. J'ai tout lu attentivement, sauf le code source, où j'ai globalement plus d'expérience et m'estime plus capable de faire à ma propre sauce. Tout paraît clair et faisable. Je ne me lance pas dans des choses compliquées comme articuler le robot, ou lui faire tourner la tête. Si cela vous tente, de tels projets plus avancés existent [WALL2].

Facile ? En théorie, c'est l'affaire de deux ou trois semaines. Dans la réalité, j'y suis depuis plus de 8 mois. Je suis allée de difficultés en difficultés : j'en ai rencontrées là où je n'aurais jamais imaginé qu'il puisse y en avoir. Et j'ai appris. Cet article est le récit de mon aventure.

-- Le stringing --

Le premier problème auquel je me confronte est un fort "stringing" de la part de mon imprimante 3D. Elle imprimait bien, puis s'est progressivement mise à laisser couler des petits filaments lors de ses déplacements, "cheveux d'ange". Cela s'appelle le *stringing*.

Ma buse étant vieille et complètement émoussée, je la change, ainsi que le corps de chauffe. Cela va mieux, mais pas assez, le stringing est encore présent. J'accuse tout de suite l'humidité de ma bobine. Elle est vieille (largement plus d'un an) et entreposée dans un garage sans grande précaution. Je découvre qu'il existe des armoires à atmosphère contrôlée, des séchoirs à bobines [SEC] etc. J'opte pour une solution simple : monter la bobine à l'étage, où il fait plutôt trop sec. Pendant qu'elle sèche, je sonde les personnes de mon Hacker Lab quant à leur bobine de PLA préférée, en restant à des prix raisonnables. J'acquiers une bobine eSUN, et je réimprime.



C'est beaucoup mieux, mais le stringing est quand même là. Je suis tenace, car je sais que j'ai déjà eu de bien meilleures impressions. Je me documente [RETRAC]. A ce stade, j'aurais pu écrire un article complet sur le stringing sous toutes ses formes :) Cela peut provenir de la distance et de la vitesse de rétractation du fil notamment : si le filament continue à couler pendant que la buse se déplace, on obtient ce fameux fil inesthétique. Je joue avec la configuration, mais c'est sans effet.

Au hacker lab, on me parle alors de température d'impression. Sur la boîte de mon filament, il est marqué que je dois imprimer vers les 210°C. Mais :

1. Est-ce que je suis vraiment à 210° lorsque l'imprimante croit y être ?
2. Cette température est une recommandation générale. Dans la pratique, chacun imprime à la température adéquate pour son imprimante, le filament et la pièce où on se trouve.

Pour trouver cette température, une pratique simple est l'impression d'une "tour de température" [TEMP]. Chaque étage de la tour est imprimé avec une température croissante, par paliers, du genre de 180 à 230°. A l'impression, on voit très nettement la meilleure température pour son cas. C'est donc ce que je fais, et ô surprise, la meilleure température pour moi est vers 186°C. C'est assez bas, éloigné des 210°C : je n'avais jamais tenté d'imprimer à cette température alors que cela fonctionne à merveille.

Au passage, quand même, j'en profite pour contrôler le PID de la température [PID] de mon imprimante. Le PID est l'algorithme qui régule la température de la buse et du plateau. S'il n'est pas bon, cela peut poser des problèmes comme le stringing (ou des problèmes plus graves). Mais tout va bien, ouf.

J'ai donc maintenant des pièces qui s'impriment bien, sans stringing, et de bonne qualité. Je suis contente d'être allée jusqu'au bout de cette étape,

de ne pas avoir abandonné en me disant "oh, il y a un peu de stringing, c'est pas grave, je poncerai". En m'y penchant, j'ai beaucoup appris, et je suis maintenant bien mieux armée pour faire face à des problèmes similaires.

-- Les chenilles du wall-E --

En guise de pieds, wall-E est affublé d'une chenille de chaque côté de son corps. Les chenilles sont constituées de bandes étroites et assemblées sur leur côté long. Pour éviter d'imprimer une centaine de bandes, le concepteur a directement pré-assemblé une vingtaine de bandes d'un seul coup et on imprime cette plaque pré-montée. Très bonne idée. Chaque "pied" est fait de deux plaques qu'on accroche l'une à l'autre pour faire une chaîne plus longue. De petites attaches de la première plaque viennent se loger dans les encoches de la 2ème plaque. Cela permet de les relier.

Sur la vidéo de Jeje95, cela marche très bien. Très honnêtement, je ne sais pas comment il a fait. Moi, c'est impossible. Ces attaches sont trop petites : elles se tordent. Pareil, les encoches sont petites et comme on ne peut pas utiliser de support d'impression à ces endroits, le PLA "goutte" un peu, et l'impression n'est pas totalement précise. Du coup, les attaches ont du mal à rentrer. J'en rentre une, l'autre sort. Je bataille, je casse des attaches, je réimprime, je re-casse etc.

Je tente une impression non pas en PLA mais en TPU. C'est un filament fait pour les parties flexibles. Je me dis qu'il conviendra bien pour des chenilles, et que, peut être, il sera plus facile de relier les bandes en TPU. Cependant, l'impression de TPU est plus compliquée que celle du PLA : mes bandes ont bien cet aspect souple que je recherche pour des chenilles, mais la précision est moins bonne, donc les attaches et encoches sont moins précises et au final j'abandonne ce matériau.

Je suis prête à passer les chenilles par la fenêtre. J'expose le problème à mon cher et tendre qui regarde, essaie, et décrète que ça ne marchera jamais, qu'il faut opter pour une autre solution. Il me propose de relier les deux plaques par du fil de fer, en perçant des trous aux points d'attaches. C'est un peu moins beau – quoique cela donne un aspect "récup" qui convient très bien à wall-E, puisque c'est un robot poubelle. – mais c'est très solide.

-- D'autres péripéties d'impression --

J'ai oublié de dire que les chenilles s'enroulent autour d'une structure d'engrenages. Ces engrenages sont censés entraîner la chenille. C'est un peu serré, c'est fait exprès car sinon la "chenille va flotter" et sortir de son emplacement. Mais ce n'est pas facile à assembler non plus. A force de tirer par ci et par là, évidemment, je casse des engrenages. N'allez pas croire que j'ai deux mains gauches. Sans me vanter, j'ai l'habitude de faire des maquettes et du travail assez minutieux. Il faut donc les réimprimer. Si vous n'avez jamais imprimé en 3D, une impression c'est toujours lent. Rares sont les pièces qui s'impriment en moins d'une heure. Les impressions prennent vite 1 ou 2 heures pour des pièces d'une dizaine de centimètres carrés.

La même histoire se répète sur la tête de wall-E : lorsque j'insère

la tête dans le cou, paf, la pièce se casse. Autant réimprimer un engrenage prend moins de 2 heures, autant la tête est un gros élément qui nécessite plus de 8 heures d'impression. Plutôt que de réimprimer,

et aussi parce que le système d'assemblage ne paraît pas très rusé / solide, mon conjoint me propose de percer et visser le cou dans la tête : simple, mais beaucoup plus solide.

Enfin, je tais quelques petits problèmes de précision sur l'assemblage du corps de wall-E. Le corps est un cube, assemblé côté par côté. Mes impressions sont plutôt bonnes, mais tout de même, sur certains côtés, il faut que je ponce pour que cela s'ajuste bien. Poncer, c'est la seconde nature des gens qui font de l'impression 3D ! Il y a les erreurs bêtes aussi, du genre j'ai imprimé deux jambes gauches (j'appelle "jambe" la connexion entre le corps de wall-E et sa chenille).

Ces déboires sont assez bénins, mais ils prennent du temps. Ils laissent également planer une interrogation : pourquoi est-ce que rien de tout ça n'est montré sur la vidéo d'assemblage ? Suis-je vraiment bête (aïe) ? Je ne vois pas comment éviter certains des problèmes que j'ai rencontrés - à mois de posséder une imprimante vraiment haut de gamme...

-- Parlera-t-il ou ne parlera-t-il pas ? --

Au niveau de la programmation, je suis plus à l'aise donc je peux plus facilement voler de mes propres ailes. A la place de l'ESP32 prévu par Jeje95, j'opte pour un vieux Wemo D1 mini [WEMO] parce que je l'ai en stock.

Prudente, je commence à câbler et tester les composants un à un. Notamment, je n'ai jamais utilisé le player MP3-TF-16P [MP3], qui permet de jouer des MP3 stockés sur une carte SD. Je relie le composant à mon wemo et à un haut parleur. Je récupère des sons de wall-E et je les mets sur la carte micro SD en respectant les noms de répertoires : les fichiers doivent être mis dans un répertoire "MP3" et s'appeler 0001.mp3, 0002.mp3 etc.

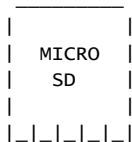
J'essaie de jouer les sons, mais le lecteur n'est même pas reconnu :(

```
+ -----+
| Initializing DFPlayer ... (May take 3~5 seconds) |
| Unable to begin:                                |
+ -----+
```

Il me faudra bien trop de temps pour réaliser que je me suis trompée sur le schéma de câblage, que RX et TX ne vont pas du tout au bon endroit...

Je rectifie, cette fois-ci mon player est reconnu, mais pas moyen de jouer les MP3. J'y passe un bon moment, je ne vois aucun problème. Je finis par incriminer le composant en lui même et tester sur un autre MP3-TF-16P: même résultat. Je teste de nombreuses librairies (DFRobotDFPlayerMini, DFPlayerMini_Fast...), je rajoute un peu de temporisation entre deux requêtes au cas où ça aille "trop vite" pour le player, je re-formatte la carte SD en testant d'autres casses pour les répertoires et fichiers: mp3, 0001.MP3... Rien n'y fait.

Là, je me souviens d'un vieux problème qu'un créateur de challenge avait rencontré pour ph0wn CTF : son challenge ne fonctionnait qu'avec des cartes micro SD *8G*, pas avec des cartes plus grosses comme 32 ou 64G. Je regarde donc ma carte : non c'est bien une petite 8G, mais j'ai une carte 16G: si je testais avec ? Et voilà, c'est la solution ! Ma carte 8G n'était pas reconnue (timeout), tandis que la 16G fonctionne.



Je teste à nouveau : j'obtiens un son, mais pas moyen de jouer tous les sons à la suite les uns des autres : mon programme n'arrête pas de rebooter. J'inclus de bêtes printf dans mon code pour mieux comprendre ce qui se passe. C'est horrible à déboguer, car ça reboote intempestivement, pas toujours au même endroit et je me lance dans de fausses corrélations sur mon code et les crashes. Puis, au bout d'un moment, j'ai un sursaut d'intelligence (ben oui, ça sert...) : si c'était juste un problème d'alimentation ? Et oui, j'alimente mon équipement via une rallonge USB fixée à un hub USB sous forme de R2D2. Oui, oui, ça va, arrêtez de rigoler ;-)

L'objectif est d'éviter de connecter mes devices USB sous le bureau, car ma tour est dessous. Mais évidemment, c'est un très mauvais choix en terme d'alimentation pour des composants comme les Arduino, Raspberry Pi etc. Je connecte donc directement mon matériel au port USB de ma tour, et là, miracle, ça joue nickel.

-- Exit le wemo --

Je décide de passer au test de l'écran LCD. Oh la la : ça fait un paquet de fils sur un breadboard ça. C'est là que je réalise en comptant tous les GPIO qu'il me faudra qu'il en manque (de peu) sur le wemo. Je ne vais pas pouvoir les inventer, je vais devoir changer de composant. N'ayant rien d'autre en stock avec plus de pins, j'achète un ESP32-WROOM32-32. Je porte mon code soigneusement codé pour ESP8266 sur ESP32: les librairies changent, l'interface n'est pas tout à fait la même, mais ça se passe bien, et assez vite.

Ça marche : j'arrive à afficher à l'écran. Je donne un peu libre court à mon imagination pour afficher des Pico le Croco, le logo de Ph0wn, le logo du SHL (mon hacker lab). Je m'amuse un peu :) J'ajoute le player MP3 et le haut parleur : ça marche toujours. Voici où j'en suis actuellement : le robot est terminé en terme d'impression, il faut un jour que je m'arrête d'ajouter des gadgets inutiles dans mon code, et que je teste l'aspect tactile de l'écran (je n'ai pas encore câblé cette partie). Ensuite, il faudra souder l'ensemble sur une plaque de prototypage plutôt qu'un breadboard, mettre dans le corps du wall-E, refermer et profiter :)

-- Conclusion --

Mon wall-E sera "bientôt" terminé (enfin, il faut que je dégage un peu de temps pour m'y plonger), et je l'exhiberai avec plaisir. Je suis certaine

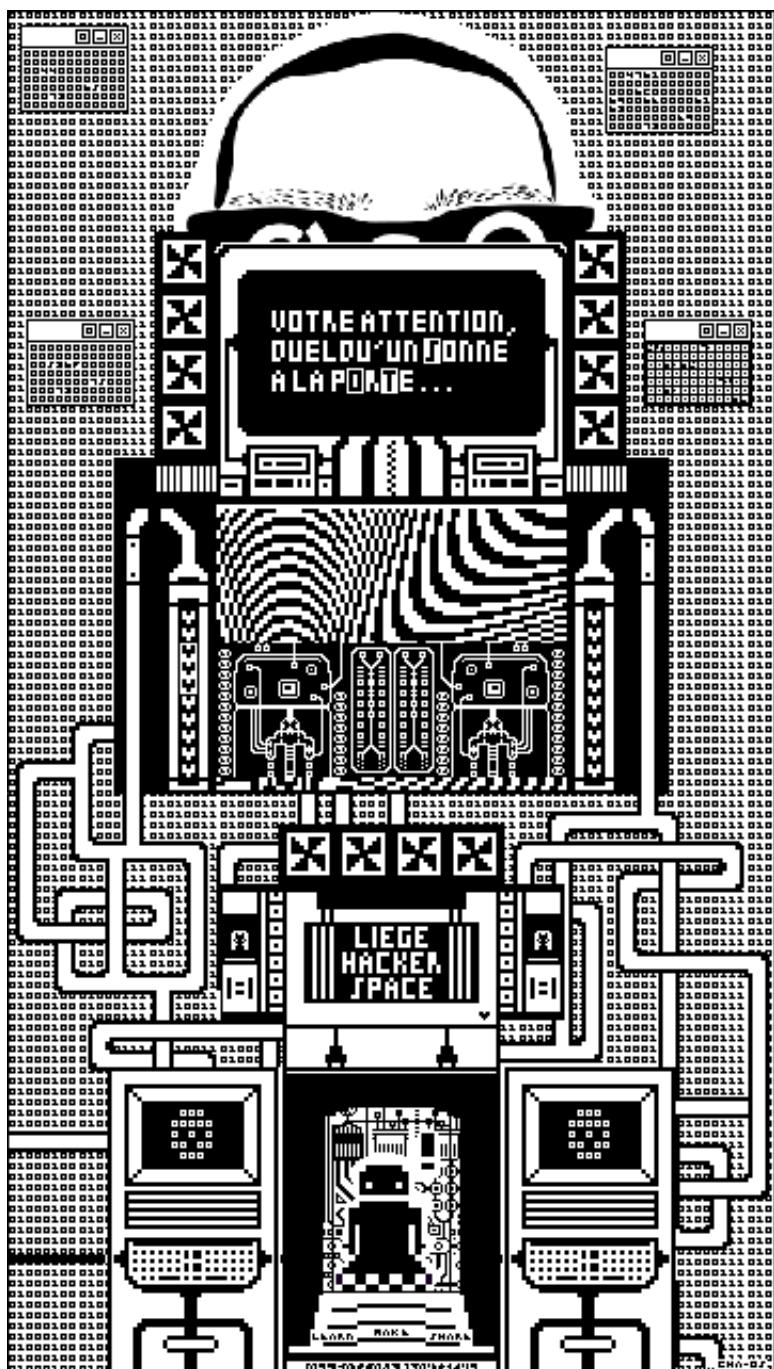
que, même s'il fera son effet, et que je glousserai de joie devant les "aaah" et les "oooh" des amis, aucun d'eux ne réalisera le temps que cela m'a pris. Un temps tout à fait déraisonnable pour un objet "qui ne sert (quasi) à rien". C'est clairement le *chemin* qui compte. Je n'ai pas seulement monté un wall-E, j'ai surtout appris et passé du bon temps. Ceci n'est pas réservé aux projets de Hackerspace d'ailleurs. On retrouve le même principe avec les fameux "write-up" de CTF. Ils ne nous montrent (généralement) que le chemin gagnant. C'est oublier le fait qu'on passe une grande partie du temps à se tromper, recommencer, réessayer, sans parler des effets de la fatigue après 6h de compétition.

Vous avez devant vous une vidéo de 10 minutes qui vous explique comment réaliser votre projet ? Soyez lucides : les vidéos montrent rarement le revers de la médaille, vous mettrez donc beaucoup plus de temps. Mais si la partie "bricolage" vous plaît, si vous aimez expérimenter, tester, re-tester, alors foncez. Au final, c'est un passe-temps intéressant et peu coûteux.

-- Cryptax

[VIDEO] <https://www.youtube.com/watch?v=2QGqMb3aLg4>
[WALL1] <https://www.thingiverse.com/thing:6578977>
[JEJE] <https://jeje-linge.fr/en/products/alarm-clock-and-wall-lamp>
[CODE] <https://github.com/jejelinge/WALL-E>
[WALL2] <https://www.thingiverse.com/thing:3703555>
[SEC] <https://all3dp.com/fr/2/secher-filament-pla-abs-nylon/>
[RETRAC] <https://www.youtube.com/watch?v=GXSqZ68UdsE>
[TEMP] <https://www.youtube.com/watch?v=N4Yt4tmFxho>
[PID] <https://teachingtecht.github.io/calibration.html#pid>
[WEMO] https://www.wemos.cc/en/latest/d1/d1_mini.html
[MP3] <https://www.instructables.com/Tutorial-of-MP3-TF-16P/>

-- EOF --



Fuck around, Mess Around, 10 ans de *GB0*, un Grand Bordel Organisé

Cha-0s & iooner
Liège Hackerspace (LgHS)

-- CONTENT --

Le Liège Hacker Space est sis 16 bis rue de la Loi, dans le quartier d'Outremeuse [!], à 4020 Liège, wallifornie, Gelbique.

On y trouve des humanoïdes, des chaises roulantes, en état ou pas, du café, du miel (en saison), des bières, du matériel mutualisé comme des imprimantes 3D, des minitels, une photocopieuse, tout un tas de brol(s) et trigus, en état ou pas, des gauff', des projets, foireux ou pas, morts, vivants, ou mort-vivants, des collab's in & out, des individualités, des idées, des techniques, des idées technico-politiques, des valeurs, du fun, du moins fun, des dramas, des gens qui partent, qui viennent, qui reviennent, quelques dingeries, de la vie, de la mort, une boucle inconditionnelle !

C0n-tak' : ping at lghs.be
Inf0s : hxxps[://]lghs[.]be/

LgHS feat Lilit

Le Liège Hacker Space héberge aussi l'asbl Liège Linux Team pour les Linux Meeting Party qui ont lieu en soirée une fois par mois, ainsi que des Linux Install Party en mode un peu plus random.

Inf0s : <https://www.lilil.be/>

Do you wanna collab ?

Envie de passer nous faire coucou ? De monter un projet commun ?
De discuter d'un truc technique un peu exotique ? De voir comment on
fait de nos toilettes un havre de pets (de PAIX bordel, de PAIIIX) ?
Envoyez nous un mail, une lettre, même un...FAX (sorry, c'était
seulement pour la ref' cinématographique)

Do what you want cauz' a ~pirate~ is ~free~

Sommes-nous des pirates ?

Non.

Si tu te perds au zinc de Robert, tu verras que l'entrée 'pirate'
correspond à la fois à un nom et à un adjectif [1].

Au sens 'propre' et 'ancien' : 'Aventurier qui courait les mers pour
piller les navires'.

Au sens 'propre' et 'moderne' : 'Personne qui attaque un bateau pour le
piller'.

Le seul truc qu'on 'attaque' pour 'piller' c'est les coquetels de 15
août de nos voisins de la boulangerie "sans patron". Mais dans le fond
on n'attaque rien, on échange [share], le space avait réparé la fissure
de leur container à rouleeeeeettes [make]. Pour le pillage, on préfère
parler de 'pliage' quand il s'agit de définir l'état dans lequel
certains d'entre nous sont rentrés (à pieds et sans vomir [learn]).

Donc, nous ne sommes pas des pirates au sens 'propre', qu'il soit
'ancien' ou 'moderne'. Alors, qu'en est-il du sens 'figuré' ?

'Individu sans scrupules, qui s'enrichit aux dépens d'autrui'.
(-> escroc).

Clairement non, personne ne s'enrichit aux dépens de personne et
l'asbl par définition, n'a pas de but lucratif.

Néanmoins, depuis 2018 et la réforme du droit des entreprises,
les asbl ont été 'assimilées' à des 'entreprises', ce qui a créé de
nouvelles obligations légales et contraintes variées (inscription à la
BCE, assujettissement à la tva, taxes,...). L'asbl doit aussi
rendre des comptes à l'état. (pleins d'histoires)

Robert parle également de 'pirate de l'air, personne qui détourne un
avion ou menace la sécurité des passagers pour exercer un chantage'.

On ne fait pas ça, même pas le dimanche, même pas 4 ze fun.

Les seuls pirates de l'air qu'on a, c'est ceux qui utilisent les toilettes après le repas collectif du mercredi...

Nous ne nous sentons pas concernés par cette dénomination.

Ensuite, 'pirate informatique, qui s'introduit dans un système informatique (-> crackeur, hackeur)'.

Au space, on ne s'introduit pas sans autorisation (!), dans un système informatique, même si en Belgique, on est 'autorisé' à certaines choses pour autant de respecter un 'cadre légal' qui sent le souffre [2].

Bref, toussa, cépanou, mais, du coup.

Sommes-nous des 'crackeurs' ?

Non, personne ne consomme du crack dans ou en dehors du space et nous n'avons aucune intention criminelle.

Sommes-nous des 'hackeurs' ?

Jouer, ça nous tient à coeur et on agit sans intention de nuire. Robert nous susurre à l'oreille à la façon d'un oncle un peu trop aimant, qu'il recommande d'utiliser le terme 'fouineur'. Fouineur ? Mmmh...

Sommes-nous des 'fouineurs' ?

:) Man the man, man <3 (autre histoire).

Enfin, l'adjectif renvoie à 'Clandestin, illicite. Radios pirates (en italique dans le texte)'. Il y a des radio-freaks au space, sous licence. Nonobstant, le concept de 'clandestin' ne fait pas non plus, unanimement sens (autre histoire).

T'as trouvé ça décousu ? Attends la suite !

Sommes-nous ~libres~ ?

Bref retour d'expérience sur 10 ans d'admin de hackerspace.

Dix ans à jongler entre la liberté prônée par les hackerspaces et la réalité du quotidien. Prévention incendie, accidents, déclarations, factures, voisinage, assurances, cambriolage, déménagements et dramas.

Le mythe du hackerspace libre et autogéré :

"Viens, fais ce que tu veux"

La réalité : c'est surtout "Viens, fais ce que tu veux, MAIS..."

Le hackerspace de Liège, c'est un petit miracle d'équilibre entre chaos

et structure, un con-promis permanent entre autonomie et contraintes.

Le discours "Viens, fais ce que tu veux" cache souvent un travail invisible des gens qui assument la responsabilité collective. La liberté des uns repose sur un noyau de servitude volontaire (de 'dévouement') des autres.

Derrière cette liberté organisée se cachent d'autres ombres : les fantômes de la do-cratie. La gouvernance c'est chiant, elle apparaît quand le collectif et les enjeux grandissent, que les visages changent, que la mémoire se dilue, que la confiance ne suffit plus.

La gouvernance, ce n'est pas le trône que vous imaginez, c'est un égout. Tout comme l'égout, elle n'est remarquée que par l'odeur...

"Qui fait, quoi, qui décide, comment ?".

Grandes questions car, tout le monde ne peut ni ne veut "faire". Factuellement, il y en a qui font, d'autres qui ne font pas. L'autorité vient du faire, pas de la couronne. Elle disparaît à la fin de l'action, des exceptions, actions irréversibles, du structurel et des obligations légales.

Dix ans plus tard, on est loin de la recette parfaite et le qui-quoi-comment sont sans cesse réinterrogés, redessinés.

Des conseils d'administration sont tombés, de nouveaux membres sont arrivés, d'autres sont partis. Il y a les fidèles du premier jour et aussi des déçus. On a juste appris à réparer, encore et encore.

C'est peut-être ça, la vraie liberté : un système qui tient debout parce qu'il n'a pas peur de tomber.

Au Liège Hackerspace, après un déménagement, des travaux interminables et un cambriolage marquant, nous préparons aujourd'hui notre prochaine décennie. Avec elle, une tentative d'un nouveau modèle de gouvernance que nous espérons pouvoir vous détailler ici, dans les prochains numéros ou sur notre wiki un peu décrépi.

Ne dit-on pas que faire et défaire, c'est toujours faire ?

[1] <https://dictionnaire.lerobert.com/definition/pirate>

[2] squiiik
& tatatum temporis !

<https://www.ejustice.just.fgov.be/eli/loi/2024/04/26/2024202344/justel>

-- EOF --

surveillance et narcotraficotage

Noë Flatreaud <nflatrea@mailo.com> (Beemo)
GRENOBLE HACKERSPACE (Grenoble)

-- CONTENT --

//

Oui, je recycle mes articles!
Z'allez faire quoi, le lire ? Essayez donc, z'êtes mêmes pas cap!
Vous pouvez retrouver le lien original par ici [^1]

//

-- 05 Mar, 2025

Depuis plusieurs années, nous assistons à une dérive sécuritaire dans de nombreux pays. Les gouvernements justifient des atteintes aux libertés individuelles par des discours alarmistes sur la criminalité, le terrorisme et d'autres menaces.

En France, cette tendance s'est intensifiée sous le gouvernement actuel, avec, en plus des réduction de droits sociaux, et des passages en forces, des lois restreignant progressivement notre droit à la vie privée et à la défense.

Retaillaud en est le plus pur exemple, récemment désigné comme ministre de l'intérieur, membre ultra-conservateur du mouvement pour la france [^2] puis des républicains, ne cesse de vouloir détruire les avancées sociales de notre pays.

La récente proposition de loi sur le « narcotrafic » avancée au Parlement

représente une menace sérieuse pour nos droits fondamentaux. Ce texte, bien qu'initialement présenté comme une réponse à la lutte contre le trafic de drogues (rien que ça c'est problématique), va bien au-delà et ouvre la voie à des mesures de surveillance intrusives (vidéo surveillance algorithmique et Chiffrement) contre-nous.

Cette dernière introduit des mesures portant atteinte à la protection des messageries chiffrées (Whatsapp, Signal...), permettant aux autorités d'accéder à nos conversations privées. Elle renforce également les capacités de surveillance de la police, autorisant l'activation à distance des micros et caméras de nos appareils connectés. De plus, le « dossier coffre » empêche les citoyens de connaître les modalités de leur surveillance, sapant ainsi le droit à une défense équitable.

Les implications de cette loi vont bien au-delà du simple trafic de drogues. Elle pourrait (et très franchement elle l'est déjà un peu) être utilisée pour cibler des militants, des journalistes et toute personne s'opposant à l'ordre établi. En élargissant le cadre de la criminalité organisée, le gouvernement cherche à justifier des mesures de surveillance qui devraient être réservées à des situations d'exception.

La lutte pour la protection de nos libertés individuelles est un combat qui nous concerne tous.

Nos vieux luttaien déjà pour légaliser chiffrement pour un usage civil [A3], alors pourquoi revenir en arrière ? Le chiffrement nous protège nous, et tous ceux désirant s'exprimer librement, sans crainte de répression.

"Quand vous avez des choses à cacher, c'est évident, vous passez d'abord par le chiffrement" - Bruno Retailleau, (Ex-)Ministre de l'Intérieur

>> C'est parce qu'on a tout à protéger qu'on passe par le chiffrement [A4,5,6]

Biz.

-- Références

[A1] <https://nflatrea.bearblog.dev/surveillance-et-narcotraficotage/>

[A2] https://fr.wikipedia.org/wiki/Mouvement_pour_la_France

[A3] <https://www.bortzmeyer.org/crypto.pdf>

[A4] <https://www.laquadrature.net/>

[A5] <https://www.laquadrature.net/narcotraficotage/>

[A6] <https://video.lqdn.fr/w/kNaBnnbV97mwVpsL8jfBbQ>

-- EOF --

DISCLAIMER

Ce zine est une oeuvre collective, à but informatif, écrite par des gens qui savent (parfois) ce qu'ils font. En aucun cas ils ne peuvent être tenus responsables de vos bêtises.

-- INFO --

9600 bauds est un ezine inter-hackerspaces francophones à l'initiative de GRENOBLE HACKERSPACE. Vous pouvez partager vos projets, vos actualités, vos réflexions, vos tutoriels, vos retours d'expérience, ou tout autre contenu de votre communauté dans un format volontairement minimaliste et nostalgique

SOMMAIRE

00.TXT : Sommaire et Présentations Staff 9600
01.TXT : Radio Libre, radio pirate! Beemo
02.TXT : RADIO TOC TOC : une sonnette pour ton bat' adada
03.TXT : Hppchkngrf: C&Embedded quick refs tixle geek
04.TXT : wall-E: sueur, fils et plastique fondu cryptax
05.TXT : Fuck around, Mess Around, 10 ans de *GBO* Cha-0s & iooner
06.TXT : Surveillance et narcotraficotage Beemo

-- CONTACT --

Tu peux nous contacter via :

- IRC: #9600 sur irc.libera.chat 6667
- EMAIL: Staff 9600 <9600@ghspace.fr>

Besoin de confidentialité ?

-----BEGIN PGP PUBLIC KEY BLOCK-----

```
mDMEaMau5BYJKwYBBAHArw8BAQdAIrisQTVU+voZh3o13A51ZpOz0Hf4a3ie5dc
0TAcQTy0HFN0YWZmIDk2MDAgPDK2MDBAZ2hzcGFjZS5mcj6IkwQTFgoAOXyHBDXY
Zn1Hx05Ta/BFq3t6TmyI4pAlBQJoxq7kaHSDBQsJCACCAiICBhUKCQgLAGQWAgMB
Ah4HAheAAAAoJEht6TmyI4pAl4rUA/2RhQ1zFNjG1sbpYn2yhd4vw6ELWv3jbsbV
k9hXhFAYAP436htohxy36vUY/1ZAashInBXD3ThvcfgbGZ04rfy7BLg4BGjGruQS
CisGAQQB1lUBBQEBB0BXqgHHKxM0ZPHJty2Hzn8MGL5Mn1R6BiAgBCDqWl22UgMB
CAeIeAQYFgoAIBYhBDxYZn1Hx05Ta/BFq3t6TmyI4pAlBQJoxq7kaHsMAAoJEht6
TmyI4pAl0g4A/2/eb+Y/CwJhIJ8WUS3ftNw6Yi929SBizt9JAU5evpACACyU6ku
/U2Zk/Ia96Lc3J0CoresASYjvc2vj+7V9uVAQ==
=RVqt
```

-----END PGP PUBLIC KEY BLOCK-----

-- EOF --

9600@ghspace.fr
#9600 sur irc.libera.chat